Lab 05 and Lab 06

1. Introduction to React JS

React is a declarative, component-based JavaScript library for building fast and interactive user interfaces. Developed by Facebook (Meta) in 2013, it powers applications like Facebook, Instagram, Netflix, and Airbnb.

Key Features:

- **Virtual DOM**: Diffing algorithm for efficient updates.
- **Component-Based Architecture**: Reusable, self-contained pieces.
- **JSX**: JavaScript + XML syntax for UI.
- **One-Way Data Flow**: Predictable state management.
- **Hooks** (React 16.8+): Functional components with state/effects.

Why React? High performance, huge ecosystem (React Router, Redux, Tailwind, etc.), and excellent developer experience.

Prerequisites: HTML5, CSS3, Modern JavaScript (ES6+), Node.js ≥18.

Modern Approach (2026): We use **Vite** (fastest bundler) + Functional Components + Hooks. Create-React-App (CRA) is deprecated.

Topic 01: Setting Up a React Project

Objectives: Install Node.js, create project with Vite, run dev server, understand basic scripts.

Theory: Vite uses native ES modules for lightning-fast HMR (Hot Module Replacement). It replaces Webpack/CRA for new projects.

Step-by-Step Procedure:

1. Download & install Node.js from nodejs.org (LTS version recommended).
2. Open terminal / VS Code terminal.
3. Run:

```
npm create vite@latest my-react-lab -- --template react
```

4. Choose:
   - Project name: my-react-lab (or press Enter)
   - Framework: **React**
   - Variant: **JavaScript** (or TypeScript for advanced)
5. Navigate & install:

```
cd my-react-lab
npm install
```

6. Start development server:

```
npm run dev
```

   Open http://localhost:5173 (Vite default port).

Understanding Scripts in package.json:

JSON

```
{
  "scripts": {
    "dev": "vite",           // Development with HMR
    "build": "vite build",   // Production build (dist folder)
    "preview": "vite preview"
  }
}
```

First Look at App: Edit src/App.jsx → Save → Instant update in browser (HMR).

Practice Questions for Lab 1:

1. What happens if you forget npm install?
2. Create a new project named todo-app and run it. Take screenshot of browser.
3. Change Vite port to 3000 (hint: edit vite.config.js).

Topic 2: Understanding Project Folder Structure

Objectives: Explore default files, learn best practices for scaling.

Default Vite + React Structure (2026):

```
my-react-lab/
├── public/                  # Static assets served directly
│   └── vite.svg
├── src/
│   ├── assets/              # Images, fonts imported in JS
│   │   └── react.svg
│   ├── App.css
│   ├── App.jsx              # Root component
│   ├── index.css
│   └── main.jsx             # Entry point (ReactDOM.createRoot)
├── .gitignore
├── index.html               # Single HTML file
├── package.json
├── vite.config.js
└── README.md
```

Detailed Explanation:

- **public/**: Files like favicon.ico are copied as-is. Access via /favicon.ico.
- **src/**: All source code. Vite processes JS/JSX/CSS.
- **index.html**: Has . No other pages (SPA).
- **main.jsx**:

  jsx

  ```jsx
  import React from 'react';
  import ReactDOM from 'react-dom/client';
  import App from './App.jsx';
  import './index.css';

  ReactDOM.createRoot(document.getElementById('root')).render(
    <React.StrictMode>
  ```

```
    <App />
  </React.StrictMode>
);
```

- **Best Practices for Larger Projects (Add these folders):**

```
src/
├── components/     # Reusable UI pieces (Button.jsx, Card.jsx)
├── pages/          # Route-specific pages (Home.jsx, About.jsx)
├── hooks/          # Custom hooks (useLocalStorage.js)
├── utils/          # Helper functions
├── context/        # Context API files
└── assets/
```

Practice Questions for Lab 2:

1. Move react.svg to a new images/ folder inside src and import it correctly.
2. Create components/Header.jsx and import it in App.jsx.
3. Explain difference between public/ and src/assets/.

Topic 3: Components & JSX

Objectives: Create functional components, understand JSX.

Theory: Everything in React is a component. Functional components return JSX.

Example 1: Basic Component

```
// src/components/Greeting.jsx
function Greeting(props) {  // props later
  return <h1>Hello, {props.name}!</h1>;
}

export default Greeting;
```

Import & Use in App.jsx:

```
import Greeting from './components/Greeting';

function App() {
  return (
    <div>
      <Greeting name="Alice" />
      <Greeting name="Bob" />
    </div>
  );
}
```

JSX Rules:

- Return single parent element (use <></> fragment).
- className not class.
- {} for JavaScript expressions.
- Comments: {/* comment */}

Practice Questions:

1. Create WelcomeMessage.jsx that displays current date/time using new Date().
2. Create 3 components: Navbar, Footer, Hero – compose in App.

Topic 4: Props

Objectives: Pass data from parent to child.

Theory: Props are read-only. One-way flow.

Example:

```
// Parent
function Parent() {
  const user = { name: "John", age: 25 };
  return <Child name={user.name} age={user.age} />;
}

// Child
function Child({ name, age }) {  // Destructuring
  return <p>{name} is {age} years old.</p>;
}
```

Default Props & PropTypes (optional but good practice):

```
Child.defaultProps = { age: 18 };
// With prop-types library (npm install prop-types)
```

Practice Questions:

1. Create ProductCard that receives title, price, image as props.
2. Pass an array of students to a StudentList component.

Topic 5: State with useState Hook

Objectives: Manage dynamic data.

Theory: useState returns [value, setter]. Re-renders on change.

Counter Example:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => setCount(count + 1);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+</button>
    </div>
  );
}
```

State with Object:

```
const [user, setUser] = useState({ name: '', email: '' });
setUser({ ...user, name: 'New Name' }); // Immutable update
```

Practice Questions:

1. Build a toggle button (show/hide text).
2. Create color picker: state for background color, buttons change it.

Topic 6: Passing State & Props Between Components

Objectives: Parent ↔ Child communication.

Theory: Lift state up. Use callback props for child → parent.

Example – Lifting State:

```
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <>
      <Display count={count} />
      <Controls setCount={setCount} />
    </>
  );
}
```

Child to Parent:

```
function Controls({ setCount }) {
  return <button onClick={() => setCount(c => c + 1)}>+</button>;
}
```

Practice Questions:

1. Parent has theme state ("light"/"dark"). Pass to Child component and apply class.
2. Two sibling components: Input updates state in parent, Display shows it.

Topic 7: Forms with Separate State

Objectives: Controlled forms, multiple fields.

Theory: Each input value tied to state, onChange updates it.

Full Example:

```
function RegistrationForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: ''
  });

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Submitted:', formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={formData.name} onChange={handleChange} />
      {/* Repeat for others */}
      <button type="submit">Register</button>
    </form>
  );
}
```

Separate States Alternative (for simple forms):

```
const [name, setName] = useState('');
const [email, setEmail] = useState('');
```

Practice Questions:

1. Build login form with email & password (separate states).
2. Add validation: name min 3 chars, email format.
3. Build a Signup form same as NED university admission signup form.

Topic 8: Cards Using Static Data

Objectives: Render lists with .map().

Example – Product Cards:

```
const products = [
  { id: 1, name: "Laptop", price: 999, img: "url" },
  // ...
];

function ProductGrid() {
  return (
    <div className="grid grid-cols-3 gap-4">
      {products.map(product => (
        <div key={product.id} className="border p-4 rounded">
          <img src={product.img} alt={product.name} />
          <h3>{product.name}</h3>
          <p>${product.price}</p>
        </div>
      ))}
    </div>
  );
}
```

Practice Questions:

1. Create 6 static user cards (avatar, name, role).
2. Add "Add to Cart" button that logs product.

Topic 9: Tables

Objectives: Dynamic tables from data.

Example:

```
const users = [ /* array of objects */ ];

function UserTable() {
  return (
    <table className="w-full border">
      <thead>
        <tr><th>Name</th><th>Email</th></tr>
      </thead>
      <tbody>
        {users.map(user => (
          <tr key={user.id}>
            <td>{user.name}</td>
            <td>{user.email}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

Practice Questions:

1. Student marks table with columns: Roll, Name, Math, Science, Total.
2. Add delete button per row (filter state).

Topic 10: Navigation with React Router

Objectives: Multi-page feel in SPA.

Setup:

Bash

```bash
npm install react-router-dom
```

main.jsx or App.jsx:

jsx

```jsx
import { BrowserRouter } from 'react-router-dom';

ReactDOM.createRoot(...).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

App.jsx with Routes (v6+ syntax):

jsx

```jsx
import { Routes, Route, Link, useNavigate } from 'react-router-dom';

function App() {
  return (
    <>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </>
  );
}
```

Active Link & Navigate:

jsx

```
<Link to="/products" className="active">Products</Link>
const navigate = useNavigate();
navigate('/dashboard');
```

Practice Questions:

1. Create 4 pages: Home, Products (cards), Contact (form), About.
2. Protected route simulation with simple state check.

Final Comprehensive Practice Projects (10-12)

Build these **independently** to master React. Use Tailwind for styling (optional: npm install -D tailwindcss).

1. **Counter with History** – Increment, decrement, reset + list of past counts.
2. **Simple Todo App** – Add, delete, mark done (useState + array).
3. **User Registration + Display Cards** – Form → Add to list of cards.
4. **Product Catalog with Search** – Static products → filter by name/price.
5. **Student Marks Table with Sort & Filter** – Sort by marks, filter by subject.
6. **Multi-Page E-commerce** – Home, Products (cards), Cart (state lifted), Checkout form.
7. **Quiz Application** – 5 questions, score tracking, next/prev using state & Router.
8. **Temperature Converter** – Celsius ↔ Fahrenheit + history log.
9. **Shopping Cart** – Add/remove items, quantity, total calculation.
10. **Dashboard with Tabs** – Profile, Settings, Orders (state controls active tab).
11. **Blog Mini-App** – Static posts as cards + single post detail route.
12. **Full Integration Project** – "University Portal": Navigation + Student list (table) + Add Student form + Profile cards. Use lifting state + React Router.

Submission Guidelines:

- Clean folder structure.
- Comment code.
- Deploy to Vercel/Netlify
- Screenshot every feature working.